

KOSAKATA (bagian dari virtual memory system)

1. RAM = tempat kerja CPU
2. CPU jalanin program dari RAM
3. Alamat fisik = lokasi asli di RAM
4. Alamat virtual = ilusi buat program
5. MMU + Page Table = yang nerjemahin virtual ke fisik
6. Swap Area = ruang di hardisk atau SSD yang dipakai sebagai RAM Cadangan
7. Offset = Jarak dari awal sebuah page ke lokasi data yang mau diakses.

1.) oke yang pertama perlu diketahui adalah, Ketika kita membuka aplikasi atau sebuah program nah itu tuh bakalan disimpan dulu di hardisk/SSD (tempat penyimpanan) nah setelah itu akan dimuat ke RAM, kenapa ke RAM karena itu merupakan bagian yang mudah dan cepet di aksesnya sama CPU (Otak Komputer), jadi CPU itu akan menjalankan satu-satu dari RAM

Jadi:

program → dimuat ke RAM → dibaca dan dijalankan CPU

2.) RAM itu merupakan random access memory isinya itu adalah tempat penyimpanan SEMENTARA dan SUPER CEPAT, nah isinya itu bisa R/W (Read/Write) dengan sangat cepat. tapitapitapi dia itu bakalan ilang kalau si computer dimatiin makanya semua program aktif(yang lagi jalan ) disimpan dulu di RAM

3.) Sekarang masuk ke bagian alamat memori, nah alamat memori itu jadi di dalam RAM itu bentuknya kaya rak buku yang artinya banyak slotnya, nah tiap slotnya itu punya alamat, contoh

| -----  |             |  |
|--------|-------------|--|
| Alamat | Isi RAM     |  |
| -----  |             |  |
| 0x0000 | 12          |  |
| 0x0001 | 99          |  |
| 0x0002 | instruksi A |  |
| 0x0003 | instruksi B |  |
| -----  |             |  |

Nah terus si CPU itu kan dia otaknya, artinya yang menjalankannya nah kalau si CPU mau ngambil data dia itu bilang, "Ambilin isi dari alamat 0x0002 dong!"

Alamat-alamat ini disebut alamat fisik (karena itu posisi asli di RAM).

4.) Kok kita bisa jalanin program secara bareng-bareng si? kan tiap program itu minta memori, misalkan kita buka 3 program secara barengan, yaitu chrome, spotify, dan vscode nah semuanya minta tempat di RAM, tapi tau ga kenapa dia itu ga rebutan atau nabrak-nabrak? nah baru deh disini muncul alamat virtual

5.) Alamat virtual (Ilusi) maksudnya, kan nanti system operasi itu bakalan bilang ke setiap program yang dijalankan "Hei tenang ngejalanin programnya, anggap aja kamu punya memori dari 0x0000 sampai 0xFFFF sendiri",

nah jadi setiap program tuh disini ngerasa dia cuma satu-satunya program yang lagi pake memori/RAM sendirian padahal mah di belakang layer itu si alamat virtual ya bakalan di terjemahin ke alamat fisik yang berbeda dari RAM, nah si penerjemah ini itu Namanya MMU (Memory Management Unit),

atau kalau masih belum ngerti nih ya jadi alamat virtual itu alamat tipu-tipu (ilusi) yang dikasih ke program. Jadi setiap program mikir dia punya RAM sendiri mulai dari 0x0000, 0x0001, dst..., Padahal RAM fisik aslinya satu doang, dan dipakai bareng-bareng sama semua program. Gimana sistem operasi bikin ilusi ini? Dengan cara: program kasih alamat virtual (contoh: 0x0005). MMU (Memory Management Unit) ngintip di page table. Nanti page table bilang: "Alamat virtual 0x0005 itu aslinya di alamat fisik 0x1455 ya." Baru CPU ambil data dari alamat fisik itu.

Alamat virtual itu bisa sama di tiap program, tapi di belakang layar diarahkan ke alamat fisik yang beda-beda di RAM.

6. Page Table itu seperti peta buat bilang: "Kalau kamu mau alamat virtual 0x0001, itu sebenarnya di RAM fisik 0x0450 ya." Setiap program punya page table sendiri.

7. Swap area itu merupakan "RAM Cadangan" jadi kalau RAM sudah penuh, sistem operasi itu bakalan mindahin **sementara** bagian dari RAM ke disk (swap)

### Analogi Gampang:

Bayangin kamu punya meja belajar (RAM) dan lemari arsip (disk).

- Kalau meja kamu penuh, kamu geser buku yang nggak lagi dipakai ke lemari dulu.
- Kalau nanti kamu butuh buku itu lagi, kamu ambil dari lemari → taruh lagi ke meja.

Nah proses ini:

- Mindahin dari RAM ke disk → disebut **swapping out**
- Balikin dari disk ke RAM → disebut **swapping in**

### 8. Step-step paging dalam Virtual Memori

Kenapa sih kita butuh paging? Ya karena alamat virtual itu bisa besar dan acak-acakan jadi kalau misalkan disusun kaya gitu ke RAM, bakalan boros dan susah di atus jadi untuk solusinya bisa mecah memori jadi blok-blok kecil yang sama ukurannya biar lebih gampang ditata dan dalokasikan

### 9. Page dan Frame

- Page itu merupakan potongan kecil dari alamat virtual program (misalkan 4KB)
- Frame itu merupakan potongan kecil dari RAM Fisik (ukurannya sama dengan page)

Jadi intinya kalau virtual memori itu dibagi jadi pages kalau RAM dibagi jadi Frame

10. Page table itu kan tugasnya nerjemahin alamat virtual ke alamat fisik nah disini itu ada istilah offset, offset itu jarak dari awal sebuah page ke lokasi data yang mau di akses

### Analogi Biar Gampang

Bayangin kamu punya buku 

- Buku dibagi jadi **halaman** (pages).
- Di tiap halaman, ada **baris** (line).
- Kalau kamu bilang:  
| "Aku mau baca baris ke-3 di halaman 5."

Nah:

- **Halaman ke-5 = Page Number**
- **Baris ke-3 = Offset**

Jadi cara kerja page table itu kaya daftar isi yang bilang "Alamat virtual ini lagi disimpan di bagian RAM yang mana sih?" karena kan program pake alamat virtual tapi CPU tuh butuh alamat fisik kan, contohnya yaitu, misalkan aku punya RAM 16KB, Page Size = 4KB berarti RAM dibagi jadi 4 Frame karena menyesuaikan page size jadi ada 0,1,2,3 frame.

Nah kemudian program kira itu punya virtual memori 8KB berarti akan dibagi jadi 2 pages yaitu page 0 dan 1

 Page Table-nya misalnya begini:

| Page | Frame |
|------|-------|
| 0    | 2     |
| 1    | 0     |

Artinya:

- Page 0 disimpan di **Frame 2**
- Page 1 disimpan di **Frame 0**

Aku akses alamat virtual : 0x1003, nah langkah-langkah CPU itu :

1. Tentukan page dan offset  
 $0x1003 = \text{desimal } 4099$   
 $4099 / 4KB(4096) = \text{Page } 1, \text{ sisa } 3 \text{ kan berarti}$   
Page = 1  
Offset = 3
2. Lihat page table  
Page 1 -> ada di frame 0  
Frame 0 = alamat fisiknya 0x0000
3. Alamat Fisik = Frame awal + Offset  
 $0x0000 + 0x0003 = 0x0003$

Jadi, alamat virtual 0x1003 diterjemahkan jadi alamat fisik 0x0003

**Simplenya yaitu :**

- **Virtual address itu [page number][offset]**
- **Page table itu cari page kemudian dapet frame**
- **kemudian Alamat fisik itu Frame + Offset**

11. Page Fault itu kadang-kadang page yang diminta itu beum ada di ram masih disimpan di disk (swap area), jadi kalau page belum ada = **Page Fault**, ambil dari disk

## TEKNIK VIRTUAL MEMORI

1. Demand Paging : jadi demand paging itu, ketika program dijalankan, tidak semua data dari program itu dimasukkan ke dalam RAM. Hanya bagian-bagian yang dibutuhkan saja yang dimasukkan ke dalam RAM

Kenapa begitu? Ya agar memori tidak cepat habis dan komputer jadinya bisa lebih cepat menjalankan program karena hanya data yang dibutuhkan saja yang dimasukkan ke dalam RAM, Misalnya kamu punya sebuah program yang memerlukan 10 halaman memori (pages) untuk dijalankan, tapi **RAM hanya bisa menampung 4 halaman** sekaligus. Program akan meminta halaman-halaman yang dibutuhkannya sesuai kebutuhan, dan sistem akan mengelola memori dengan **demand paging**.

**Langkah-langkahnya:**

**Urutan Akses Halaman oleh Program:**

Program Akses Halaman: 1, 2, 3, 4, 5, 1, 6, 2

Artinya, program pertama-tama meminta halaman 1, lalu halaman 2, lalu halaman 3, dan seterusnya.

**RAM kapasitas 4 halaman**, dan kita akan menggunakan **demand paging** untuk mengisi halaman ke dalam RAM hanya ketika dibutuhkan.

**Proses Demand Paging:**

1. **Halaman 1 diminta:**
  - o **Halaman 1** dimuat ke RAM (RAM sekarang: [1])
  - o **Page fault** terjadi karena halaman 1 belum ada di RAM.
2. **Halaman 2 diminta:**
  - o **Halaman 2** dimuat ke RAM (RAM sekarang: [1, 2]).
  - o **Page fault** terjadi karena halaman 2 belum ada di RAM.
3. **Halaman 3 diminta:**
  - o **Halaman 3** dimuat ke RAM (RAM sekarang: [1, 2, 3]).
  - o **Page fault** terjadi karena halaman 3 belum ada di RAM.
4. **Halaman 4 diminta:**
  - o **Halaman 4** dimuat ke RAM (RAM sekarang: [1, 2, 3, 4]).
  - o **Page fault** terjadi karena halaman 4 belum ada di RAM.
5. **Halaman 5 diminta:**
  - o **RAM penuh** sekarang, jadi harus mengganti halaman. Misalnya, kita menggunakan **FIFO** (First In First Out), jadi halaman 1 yang diganti.
  - o **Halaman 5** dimuat ke RAM, menggantikan halaman 1 (RAM sekarang: [2, 3, 4, 5]).
  - o **Page fault** terjadi karena halaman 5 belum ada di RAM.
6. **Halaman 1 diminta lagi:**
  - o **Halaman 1** tidak ada di RAM, jadi dimuat dari disk (RAM sekarang: [1, 3, 4, 5]).
  - o **Page fault** terjadi karena halaman 1 baru dimuat lagi.
7. **Halaman 6 diminta:**
  - o **RAM penuh** lagi, jadi harus mengganti halaman. Misalnya, halaman 2 yang diganti.
  - o **Halaman 6** dimuat ke RAM, menggantikan halaman 2 (RAM sekarang: [1, 3, 4, 6]).

- **Page fault** terjadi karena halaman 6 belum ada di RAM.
8. **Halaman 2 diminta lagi:**
- **Halaman 2** tidak ada di RAM, jadi dimuat lagi dari disk (RAM sekarang: [1, 3, 2, 6]).
  - **Page fault** terjadi karena halaman 2 baru dimuat lagi.

Performansi Demand Paging : nah tadi kan demand paging itu fungsinya untuk menghemat memori ya, kalau performansi demand paging itu fungsinya membantu menghemat ruang memori dengan hanya memuat bagian-bagian yang dibutuhkan atau mengurangi beban RAM kalau performansi demand paging itu merujuk [ada seberapa baik atau buruk teknik demand paging dalam hal kecepatan dan efisiensi akses memori, ini itu **diukur dengan seberapa sering page fault terjadi**, yang berarti data atau halaman yang diminta tidak ada di RAM dan harus dimuat dari disk. Contohnya :

Misalkan, kita punya dua sistem dengan konfigurasi memori yang berbeda:

1. **Sistem A:** RAM-nya hanya dapat menampung **3 halaman**.
2. **Sistem B:** RAM-nya dapat menampung **5 halaman**.

Program yang dijalankan membutuhkan **5 halaman**. Dalam kedua sistem ini, kita akan mengamati jumlah **page fault** yang terjadi dan bagaimana itu mempengaruhi kinerja program.

---

#### **Skenario A: Sistem A (RAM hanya 3 halaman)**

**Urutan Halaman yang Diminta oleh Program:**

1, 2, 3, 4, 5, 1, 6, 2

- **Page fault** terjadi setiap kali program meminta halaman yang tidak ada di RAM.
- **RAM awalnya kosong**, jadi setiap kali halaman yang diminta tidak ada di RAM, **page fault** terjadi dan halaman tersebut dimuat dari disk.

#### **Langkah-langkah:**

1. **Halaman 1 diminta:** RAM [1], page fault.
2. **Halaman 2 diminta:** RAM [1, 2], page fault.
3. **Halaman 3 diminta:** RAM [1, 2, 3], page fault.
4. **Halaman 4 diminta:** RAM [2, 3, 4], page fault (halaman 1 diganti).
5. **Halaman 5 diminta:** RAM [3, 4, 5], page fault (halaman 2 diganti).
6. **Halaman 1 diminta lagi:** RAM [1, 4, 5], page fault (halaman 3 diganti).
7. **Halaman 6 diminta:** RAM [1, 5, 6], page fault (halaman 4 diganti).
8. **Halaman 2 diminta lagi:** RAM [1, 2, 6], page fault (halaman 5 diganti).

**Total page fault: 8 page fault** (dari 8 permintaan halaman)

---

**Skenario B: Sistem B (RAM bisa menampung 5 halaman)**  
**Urutan Halaman yang Diminta oleh Program:**

1, 2, 3, 4, 5, 1, 6, 2

- Sistem B memiliki **lebih banyak ruang di RAM** (5 halaman).
- Jadi, lebih banyak halaman yang bisa dimuat sekaligus tanpa harus sering mengganti halaman.

**Langkah-langkah:**

1. **Halaman 1 diminta:** RAM [1], page fault.
2. **Halaman 2 diminta:** RAM [1, 2], page fault.
3. **Halaman 3 diminta:** RAM [1, 2, 3], page fault.
4. **Halaman 4 diminta:** RAM [1, 2, 3, 4], page fault.
5. **Halaman 5 diminta:** RAM [1, 2, 3, 4, 5], page fault.
6. **Halaman 1 diminta lagi:** RAM [1, 2, 3, 4, 5], **tidak ada page fault** (halaman 1 sudah ada di RAM).
7. **Halaman 6 diminta:** RAM [1, 2, 3, 4, 6], page fault (halaman 5 diganti).
8. **Halaman 2 diminta lagi:** RAM [1, 2, 3, 4, 6], **tidak ada page fault** (halaman 2 sudah ada di RAM).

**Total page fault: 6 page fault** (dari 8 permintaan halaman)

---

**Perbandingan Performansi Demand Paging**

- **Sistem A** (3 halaman di RAM) mengalami **8 page fault**.
- **Sistem B** (5 halaman di RAM) mengalami **6 page fault**.

**Kenapa itu terjadi?**

1. Dengan **lebih banyak ruang di RAM**, Sistem B lebih efisien karena lebih banyak halaman yang bisa dimuat sekaligus, sehingga **lebih sedikit page fault** yang terjadi.
2. **Semakin sering terjadi page fault**, semakin banyak waktu yang dihabiskan untuk memindahkan data antara RAM dan disk. Ini memperlambat kinerja program.

## 2. Replacement Paging

**Page Replacement** adalah proses **mengganti** halaman lama di RAM dengan **halaman baru** dari disk **saat RAM penuh**.

Misalnya: RAM bisa simpan 3 halaman, terus program butuh halaman ke-4 → sistem harus **menggantikan** salah satu dari 3 halaman yang ada. Supaya nggak asal ganti, sistem pakai **algoritma** buat milih halaman mana yang diganti.

### - FIFO (First In First Out)

Sesuai dengan namanya jadi halaman yang paling pertama dimuat ke RAM akan dikeluarkan duluan

📦 Contoh:

Urutan halaman: 1, 2, 3, 4

RAM muat 3 halaman

| Step | Halaman | RAM       | Page Fault? | Catatan          |
|------|---------|-----------|-------------|------------------|
| 1    | 1       | [1]       | ✓           | masuk 1          |
| 2    | 2       | [1, 2]    | ✓           | masuk 2          |
| 3    | 3       | [1, 2, 3] | ✓           | masuk 3          |
| 4    | 4       | [2, 3, 4] | ✓           | buang 1, masuk 4 |

📊 Total page fault: 4

### - OPTIMAL

Buang halaman yang paling lama tidak akan dipakai lagi di masa depan. Jadi Sistem melihat **semua permintaan halaman ke depan**. Kemudian ganti halaman yang **paling lama baru akan dipakai** atau bahkan **tidak akan dipakai sama sekali**.

📦 Contoh:

Urutan halaman: 1, 2, 3, 4, 2, 1, 5

RAM muat 3 halaman

🔍 Simulasi Langkah per Langkah

| Step | Halaman Masuk | Isi RAM   | Page Fault? | Halaman yang Diganti   | Penjelasan                                                     |
|------|---------------|-----------|-------------|------------------------|----------------------------------------------------------------|
| 1    | 1             | [1]       | ✓           | -                      | Masih ada ruang, masuk 1                                       |
| 2    | 2             | [1, 2]    | ✓           | -                      | Masih ada ruang, masuk 2                                       |
| 3    | 3             | [1, 2, 3] | ✓           | -                      | Masih ada ruang, masuk 3                                       |
| 4    | 4             | [1, 2, 4] | ✓           | 3                      | 3 tidak akan dipakai lagi → diganti                            |
| 5    | 2             | [1, 2, 4] | ✗           | -                      | 2 sudah ada di RAM                                             |
| 6    | 1             | [1, 2, 4] | ✗           | -                      | 1 sudah ada di RAM                                             |
| 7    | 5             | [1, 2, 5] | ✓           | 4 (bisa juga 1 atau 2) | Semua tidak dipakai lagi, bebas pilih salah satu untuk diganti |

✓ Total Page Fault: 5



## - LRU

Buang halaman yang paling lama gak dipakai lagi. Sistem **mencatat waktu terakhir** halaman dipakai. Kalau perlu ganti, pilih halaman yang **paling lama tidak disentuh**.

📌 Contoh:

Urutan halaman: 1, 2, 3, 1, 4

| Step | Halaman | RAM       | Diganti? | Page Fault |
|------|---------|-----------|----------|------------|
| 1    | 1       | [1]       | -        | ✓          |
| 2    | 2       | [1, 2]    | -        | ✓          |
| 3    | 3       | [1, 2, 3] | -        | ✓          |
| 4    | 1       | [1, 2, 3] | -        | ✗          |
| 5    | 4       | [1, 3, 4] | ganti 2  | ✓          |

(karena 2 paling lama gak dipakai, dia yang diganti)

📊 Total page fault: 4

## PERBANDINGAN SINGKAT PAGE REPLACEMENT

### 📄 Ringkasan Perbandingan:

| Algoritma | Strategi                     | Kelebihan           | Kekurangan              |
|-----------|------------------------------|---------------------|-------------------------|
| FIFO      | Ganti yang paling awal masuk | Simpel              | Bisa buang yang penting |
| Optimal   | Ganti yang gak dipakai lama  | Paling efisien      | Gak bisa dipakai nyata  |
| LRU       | Ganti yang lama gak dipakai  | Realistis & efisien | Butuh pencatatan        |

### 3. Trashing

**Trashing** adalah kondisi **parah** dalam sistem komputer saat **terlalu banyak page fault** terjadi, sehingga **komputer malah sibuk bolak-balik ambil halaman dari disk ke RAM**, bukan ngerjain tugas programnya. Hal ini terjadi karena RAM terlalu kecil untuk menampung semua halaman yang sering dibutuhkan jadi, halaman yang baru dimuat itu sering langsung diganti lagi karena halaman lain dibutuhkan -> keluar masuk terus, akibatnya, CPU nunggu terus, karena setiap kali butuh data itu harus nunggu halaman dimuat dari disk dulu, Contohnya :

Bayangin kamu belajar pakai 3 buku, tapi cuma boleh bawa 1 buku di meja.

- Kamu buka buku A → butuh info dari B → ambil B, taruh A.
- Lalu butuh A lagi → ambil A, taruh B.
- Terus bolak-balik, kamu **sibuk ambil buku doang, gak sempat belajar!**  
Nah itulah **trashing** di komputer.

---

#### **Ciri-Ciri Terjadi Trashing:**

- Program jalan **super lambat**.
- **Page fault rate** (frekuensi halaman tidak ditemukan di RAM) sangat tinggi.
- CPU jadi **idle terus**, nunggu disk.

---

#### **Cara Menghindari:**

1. Tambah RAM.
2. Kurangi jumlah program besar yang jalan bersamaan.
3. Gunakan **algoritma page replacement** yang lebih cerdas (kayak LRU).